

A Computer Vision-Based Interactive Gesture Platform for Cognitive and Motor Development in Early Childhood

Swapnil Niraj, Sarvagya Jain, Ayush Solanki, Yash Gupta
Department of Computer Science and Business Systems
MIT World Peace University, Pune, India

Abstract—Widespread integration of touchscreen devices into the daily routines of toddlers and preschoolers has intensified scrutiny of how digital media shapes early development. Young children aged one through six are routinely exposed to non-participatory content that fails to engage the motor, sensory, and cognitive channels most receptive to stimulation at this stage of growth. To address this gap, a webcam-driven gesture interaction platform is developed that channels children’s existing screen time into structured, physically active engagement. The platform fuses the MediaPipe skeletal hand-tracking library with OpenCV-based image capture, coupling a geometry-driven gesture classifier with a Random Forest ensemble for reliable recognition of varied hand configurations. On this foundation, four purpose-built game modules are assembled: an Air Piano linking individual finger raises to MIDI chord output, a Bubble Pop activity requiring sustained pointing precision, a Gesture Drawing and Tracing module for free-form and guided digital mark-making, and a Count and Show game that validates finger-count input against displayed numerals. Internally, the system follows a four-tier design — a vision core, two independent game execution layers, and an integration shell built around a Flask server and React interface communicating over WebSocket. Testing under controlled conditions yields a gesture classification rate above 90 % and an input-to-response delay comfortably beneath 100ms. No internet access or specialized peripheral is required for operation. The work demonstrates that careful gesture-interface design can turn routine screen exposure into an experience that actively advances children’s physical coordination and cognitive readiness.

Index Terms—gesture recognition, hand landmark detection, MediaPipe, early childhood development, gamification, human-computer interaction, computer vision

I. INTRODUCTION

Digital screens now occupy a central position in the daily lives of children well before they reach school age. Usage data gathered across diverse national contexts show that children below five years devote roughly two to three hours each day to screen-based activity, and the advent of low-cost smartphones and tablet computers has caused this figure to climb steadily over recent years [1]. The dominant mode of that engagement is passive: pre-recorded video, algorithmically curated streams, and applications that respond to taps but demand nothing more sophisticated from the user. Pediatric health bodies, among them the World Health Organization, have noted that protracted

passive viewing in the pre-school years risks slowing the development of movement ability, language acquisition, and the social behaviors that form the backbone of later learning [1], [2].

The critical variable, however, is not screen exposure per se but the degree to which the interface demands genuine participation. When on-screen events are contingent on the child’s deliberate physical actions — when a gesture causes something to happen — the child shifts from spectator to operator. Educational technology research confirms that this shift carries measurable consequences: embodied digital interaction has been associated with meaningful gains in focused attention, spatial cognition, and hand dexterity [7], [8]. This body of evidence supports reframing the screen not as a passive display but as a surface that responds to the child and thereby participates in development.

The platform described in this paper operationalizes that reframing for children aged one through six. A standard webcam feeds continuous video into a hand-tracking pipeline built on the MediaPipe Hands framework [3], from which landmark geometry is extracted and interpreted by a hybrid recognition system. Four activity modules translate recognized gestures into game events, each targeting a distinct developmental capability: the Air Piano builds auditory-motor linkage and individual finger control; Bubble Pop demands sustained attentional focus and pointing accuracy; Gesture Drawing and Tracing develops fine motor coordination and visual-motor coupling; Count and Show reinforces early numeracy through finger enumeration.

Four specific contributions distinguish this work. A stratified software architecture is introduced that partitions the vision pipeline, recognition logic, game execution, and product integration into separately maintainable tiers. A computationally efficient classification scheme is described that combines closed-form geometric rules with an ensemble learner, achieving high accuracy without requiring GPU resources. Four developmentally motivated game modules are designed and evaluated against established early-childhood competency milestones. The resulting application functions without network access on hardware available in ordinary homes and early learning centers.

II. LITERATURE REVIEW

A. *Gesture Recognition in Human-Computer Interaction*

Research on gesture-driven interfaces has undergone substantial transformation since its origins in depth-camera systems that required purpose-built sensing hardware. The survey by Rautaray and Agrawal catalogued the range of image-processing strategies used for hand-pose classification up to the mid-2010s and highlighted the persistent trade-off between recognition breadth and computational cost [4]. Subsequent adoption of convolutional neural networks dramatically widened the set of gestures that could be reliably distinguished and reduced sensitivity to illumination changes. Current practice tends toward layered designs that reserve learned classifiers for configurations where simple geometric reasoning is insufficient, a strategy reflected in the present system's dual-stage recognition pipeline.

B. *MediaPipe for Hand Landmark Detection*

The MediaPipe Hands framework, reported by Zhang et al., redefined the practical boundary of real-time, CPU-only hand analysis [3]. Its architecture chains a fast palm localizer that operates on the full image with a second, higher-fidelity network that regresses twenty-one three-dimensional joint positions from the cropped hand crop. The ability to sustain this pipeline on a standard processor without accelerator hardware makes it directly applicable to commodity deployment scenarios. The vision core of the proposed platform is built entirely around this framework, consuming its normalized joint coordinate output as the entry point for all downstream gesture processing.

C. *Child-Computer Interaction Design*

Designing software that young children can operate without frustration requires departing substantially from conventions established for adult users. Hourcade's foundational review of interaction design for children documents the ways in which limited fine motor precision, shorter attentional windows, and pre-literate status impose distinct constraints on interface choice [5]. Effective designs for this demographic favor large touch or gesture targets, responses that are instantaneous and legible without text, and error-tolerant input models that accept approximate rather than exact physical input. Each of these considerations shaped the gesture threshold tuning, visual feedback immediacy, and icon-led navigation adopted in the proposed platform.

D. *Gamification in Early Learning Environments*

The systematic review by Dicheva et al. examined the documented effects of introducing game mechanics into educational software and found consistent evidence for increased engagement and session duration, particularly among younger learners [6]. Score accumulation, level progression, and reward animations were identified as the elements most reliably associated with sustained participation. Embedding instructional goals within a game loop rather than presenting them as explicit tasks appears to reduce resistance and increase willingness to retry after failure. This literature directly informed the

structure of all four platform modules, each of which wraps its developmental objective inside a feedback-rich game session rather than an explicit exercise format.

E. *Computer Vision in Educational Technology*

Vision-based sensing has been applied across a range of educational scenarios, from automated handwriting evaluation to real-time posture guidance in physical education. Yuan and Rosson evaluated a gesture-enabled learning environment with primary-school participants and documented improvements in both task engagement and assignment completion rate relative to a mouse-driven control condition [8]. Complementary investigations have targeted sign language tutoring and letter-formation assessment. A gap that the present work specifically addresses is the relative scarcity of vision-driven platforms oriented toward children below the age of six, whose interaction vocabulary and developmental needs differ markedly from those of school-age subjects studied in most prior work.

F. *Motor Skill Development Through Digital Interaction*

The maturation of fine and gross motor abilities during the first six years of life carries consequences that extend well beyond physical competence, influencing later academic performance and neurological organization [9]. Marichal et al. designed a tablet application with deliberate motor-challenge elements and measured grip strength and pointing accuracy in a preschool cohort before and after a structured interaction period, reporting statistically reliable improvements on both measures [7]. The Bubble Pop and Gesture Drawing modules draw explicitly on this evidence, structuring their interaction demands around the precise, sustained finger movements that challenge and thereby strengthen the same motor circuits engaged by traditional pencil-and-paper developmental activities.

G. *Music and Auditory Development in Early Childhood*

Anvari et al. established correlations between musical aptitude in preschool children and subsequent phonological awareness and early reading scores, suggesting that musical engagement exercises processing channels relevant to language acquisition [10]. Activities that require independent movement of individual fingers — as in rudimentary keyboard playing — additionally challenge the neural differentiation of digit control, which develops gradually across the pre-school years. The Air Piano module exploits this connection by binding each finger's extension to a distinct harmonic output, so that the child's curiosity about musical cause-and-effect simultaneously drives the fine-grained finger differentiation that motor development researchers consider a meaningful early milestone.

H. *Drawing, Tracing, and Pre-Writing Development*

The causal link between manual tracing practice and subsequent letter-recognition ability was examined by Longcamp et al., who compared children who practised letter forms by hand against those who used a keyboard and found superior recognition outcomes in the handwriting group [11]. The underlying mechanism is thought to involve proprioceptive

memory — the encoding of shape information through the kinesthetic trace of the producing movement rather than through visual inspection alone. The Gesture Drawing and Tracing module digitizes this mechanism: the spatial path of the child’s raised finger is rendered in real time on a virtual canvas, preserving the motor-production component of tracing while adding quantitative accuracy scoring as an instructional feedback layer unavailable in conventional paper-based tasks.

III. SYSTEM ARCHITECTURE

The developed platform separates its concerns across four tiers arranged in a strict dependency hierarchy. Each tier exposes a narrow, well-specified interface to the tier above it, so that any component can be modified or replaced independently without cascading disruption through the rest of the codebase. This separation also allows the four development sub-teams to work concurrently during implementation. Fig. 1 illustrates the full system architecture and the data pathways between layers.

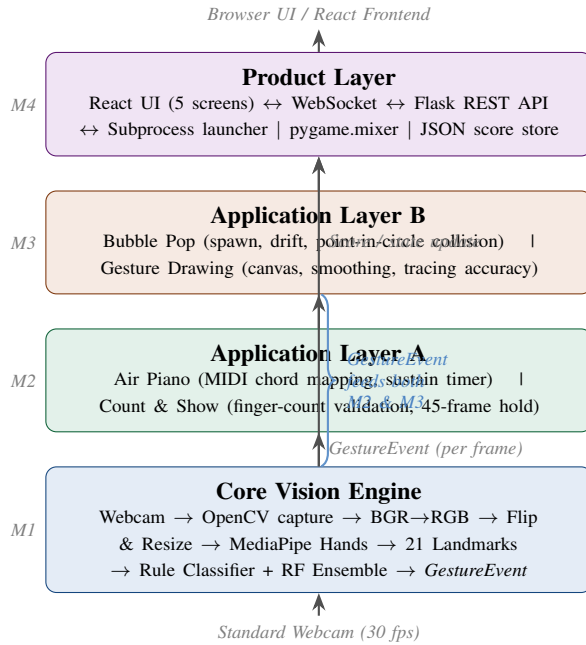


Fig. 1. Four-tier system architecture. M1 emits a structured *GestureEvent* every camera frame consumed independently by M2 and M3; state changes propagate upward to M4 for REST/WebSocket delivery to the browser.

A. M1 — Core Vision Engine

M1 is the sole tier that interacts with hardware. A continuous loop driven by OpenCV reads raw frames from the attached webcam at a target rate of thirty frames per second. Before the image is analysed, three preprocessing operations are applied in sequence: the frame is mirrored left-to-right so that children see a reflection rather than a surveillance view; the OpenCV-native BGR channel ordering is converted to the RGB sequence that MediaPipe requires; and the image is cropped and scaled to a fixed 640×480 pixel canvas to stabilise network input geometry. The conditioned image is then forwarded to the MediaPipe

Hands inference graph, which returns a set of twenty-one three-dimensional joint coordinates for each detected hand. These coordinates feed the gesture classification logic described in Section IV, which annotates each frame with a structured gesture event. The event record carries eight fields: the laterality label, the full landmark array, a five-bit finger state vector, a scalar finger count, the discrete gesture class, the screen-projected index fingertip position, the wrist screen position, and a confidence score. This record is the sole output produced by M1 and is the sole input consumed by M2 and M3.

B. M2 — Application Layer A

M2 ingests the per-frame gesture event from M1 and drives the Air Piano and Count and Show modules. For the Air Piano, M2 compares the arriving finger state vector against the vector cached from the preceding frame to identify state transitions, uses those transitions to issue MIDI note commands through the `pygame.midi` output device, and manages a frame-counted sustain timer that delays note-off messages to smooth chord decay. For Count and Show, M2 extracts the finger count scalar from the event and accumulates consecutive frames in which the count matches the currently displayed target, triggering a validated response when the accumulation reaches a threshold.

C. M3 — Application Layer B

M3 operates in parallel with M2, consuming the same gesture event stream to power the Bubble Pop and Gesture Drawing modules. The Bubble Pop game loop updates a list of active circle objects — advancing their positions, spawning replacements at the configured rate, and testing the arriving fingertip coordinate against each bubble’s boundary. The Gesture Drawing module uses the index finger position and state fields to extend a stroke on an in-memory canvas array, blending that canvas onto the live camera frame each tick.

D. M4 — Product Layer

M4 furnishes the user-visible shell that encapsulates the Python game engine within a deployable application. A Flask web server defines four REST routes through which the React frontend can start and stop game sessions, retrieve stored scores, and post new score records. A Flask-SocketIO channel carries real-time game state pushes — score increments, session status flags, and active gesture labels — from the Python backend to the browser without the latency overhead of polling. Audio beyond MIDI chord output is handled by `pygame.mixer`, and session scores are persisted to a structured JSON file so that progress is retained between launches.

E. Inter-layer Data Flow

On every camera frame, M1 emits one gesture event. M2 and M3 each receive that event and independently update their respective game states. Meaningful state changes — a score increment, a level transition, or a session-end signal — are relayed upward to M4, where the Flask server serialises and broadcasts them over the WebSocket channel to the

React client. The browser view reflects the change within the next animation frame. Non-realtime interactions such as game launch commands and historical score requests travel as discrete REST calls rather than over the persistent WebSocket connection.

IV. METHODOLOGY

A. Hand Landmark Extraction

MediaPipe Hands processes each conditioned image through a sequentially composed inference pipeline. A lightweight palm detection network first scans the complete image to locate candidate hand regions, yielding a bounding rectangle and a rotation angle for each. The cropped and rectified region is then passed to a second, more precise landmark regression model that outputs a set of twenty-one joint positions in a normalised three-dimensional coordinate frame. The anatomical mapping follows a consistent index scheme: joint zero anchors at the wrist; indices one through four trace the thumb from its basal metacarpal articulation to the distal tip; and four-index blocks thereafter describe the corresponding joints of the index, middle, ring, and little fingers in the same proximal-to-distal order. Horizontal and vertical coordinates are expressed as fractions of image width and height respectively, confining them to the unit interval, while the depth coordinate is reported relative to the wrist joint, with negative values indicating projection toward the camera. Fig. 2 depicts the full 21-landmark schema.

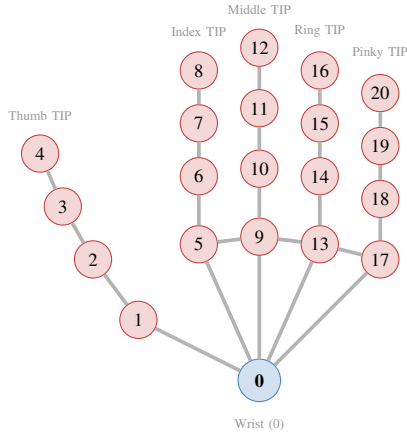


Fig. 2. MediaPipe 21-landmark hand skeleton. Each numbered node corresponds to a specific anatomical joint; nodes 4, 8, 12, 16, and 20 are fingertips used for gesture classification.

B. Rule-Based Gesture Classification

The geometry of finger extension provides a reliable, low-latency basis for classifying the most common hand configurations. Because the webcam image situates the vertical origin at the top edge with the axis increasing downward, an extended finger presents a tip joint whose vertical coordinate is numerically smaller than that of the middle knuckle. The extension state of each non-thumb digit is therefore encoded as:

$$\text{finger}_i = \begin{cases} 1 & \text{if } y_{\text{tip},i} < y_{\text{pip},i} \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

The thumb does not flex along the vertical axis and is handled separately: its extension is judged by the lateral separation between its tip and its interphalangeal joint, with the sign of the comparison reversed for left versus right hands to compensate for the anatomical mirroring introduced by the horizontal flip applied during preprocessing.

Contact between thumb and index finger — a pinch — is signalled by the inter-tip Euclidean distance dropping below a calibrated threshold:

$$d_{\text{pinch}} = \sqrt{(x_4 - x_8)^2 + (y_4 - y_8)^2} \quad (2)$$

A pinch is registered when $d_{\text{pinch}} < 0.05$ in normalised units. Aggregate hand shapes are derived from the extension vector: all five bits set indicates a spread open hand; all bits clear indicates a closed fist.

C. Machine Learning Classifier for Ambiguous Gestures

Gesture configurations that cannot be unambiguously resolved by geometric rules alone are handled by a Random Forest ensemble [12] implemented through scikit-learn. Each training sample is formed by concatenating the x , y , and z values of all twenty-one landmarks into a single 63-element vector. To remove dependence on hand position and size, each landmark is first translated so that the wrist sits at the origin, then divided by the Euclidean distance from the wrist to the base of the middle finger. At least two hundred samples per gesture class are collected in an offline recording pass and labelled manually. The fitted model is stored as a serialised binary file and loaded once during application startup; prediction at inference time then requires only a single forward pass through the ensemble, adding negligible latency to the per-frame budget.

Algorithm 1 summarises the complete per-frame logic executed by the M1 Core Vision Engine.

D. Coordinate Mapping and Canvas Rendering

All landmark positions provided by MediaPipe reside in the normalised unit square. Projection to on-screen pixel locations is achieved by scaling each coordinate by the corresponding display dimension:

$$(p_x, p_y) = (\lfloor x \cdot W \rfloor, \lfloor y \cdot H \rfloor) \quad (3)$$

with W and H denoting the render target's width and height in pixels.

The drawing canvas is stored as a zero-initialised NumPy array sharing the spatial extent of the video frame. When the index finger is raised, the projected tip position is connected to its position from the previous frame by a filled line rendered through OpenCV's drawing interface. Each rendered canvas is blended transparently over the live camera image at an opacity of 0.7 using the `addWeighted` compositing function, so that the child remains visible beneath the developing drawing.

Algorithm 1: M1 Core Vision Engine — Per-Frame Pipeline

Input : Raw BGR frame F from webcam

Output : GestureEvent $\mathcal{E}$ per frame

```

1  $F \leftarrow \text{FLIP}(F, \text{horizontal});$ 
2  $F_{rgb} \leftarrow \text{BGR2RGB}(F);$ 
3  $F_{rgb} \leftarrow \text{RESIZE}(F_{rgb}, 640 \times 480);$ 
4  $\mathcal{H} \leftarrow \text{MEDIAPIPEHANDS.PROCESS}(F_{rgb});$ 
5 if  $\mathcal{H}$  contains no detections then
6   return  $\emptyset$ ; // no hand in frame
7 foreach hand  $h \in \mathcal{H}$  do
8    $\mathbf{L} \leftarrow$  array of 21 landmarks  $(x_i, y_i, z_i);$ 
   // -- Rule-based finger state --
9   for  $i \leftarrow 1$  to 4 (non-thumb fingers) do
10     $s_i \leftarrow \mathbf{1}[y_{\text{tip},i} < y_{\text{pip},i}];$ 
11     $s_0 \leftarrow \text{THUMBRULE}(\mathbf{L}, \text{laterality}(h));$ 
12     $\mathbf{s} \leftarrow [s_0, s_1, s_2, s_3, s_4];$ 
13     $\text{count} \leftarrow \sum \mathbf{s};$ 
   // -- Pinch detection --
14     $d_{\text{pinch}} \leftarrow \sqrt{(x_4 - x_8)^2 + (y_4 - y_8)^2};$ 
   // -- Gesture classification --
15    if  $d_{\text{pinch}} < 0.05$  then
16       $g \leftarrow \text{PINCH};$ 
17    else if  $\text{count} = 5$  then
18       $g \leftarrow \text{OPEN\_PALM};$ 
19    else if  $\text{count} = 0$  then
20       $g \leftarrow \text{FIST};$ 
21    else
22       $\mathbf{f} \leftarrow \text{NORMALISE}(\mathbf{L});$  // wrist-centred,
      scale-free
23       $g \leftarrow \text{RFCLASSIFIER.PREDICT}(\mathbf{f});$ 
24     $(p_x, p_y) \leftarrow (\lfloor x_8 \cdot W \rfloor, \lfloor y_8 \cdot H \rfloor);$ 
25     $\mathcal{E} \leftarrow \{h, \mathbf{L}, \mathbf{s}, \text{count}, g, (p_x, p_y)\};$ 
26    emit  $\mathcal{E}$  to M2 and M3;
```

Landmark tracking exhibits frame-to-frame jitter that, if rendered directly, produces ragged strokes. An exponential moving average suppresses this noise:

$$\hat{p}_t = \alpha \cdot \hat{p}_{t-1} + (1 - \alpha) \cdot p_t \quad (4)$$

Setting $\alpha = 0.7$ retains continuity with the recent trajectory while admitting new position measurements at a weight that prevents abrupt jumps without introducing perceptible lag on deliberate movements.

V. MODULE DESIGN AND IMPLEMENTATION

A. Air Piano

The Air Piano module overlays a schematic keyboard graphic onto the live camera feed and assigns a distinct MIDI chord

to each of the ten fingers. Chord identities are stored in a two-level dictionary keyed on hand laterality and finger index, with each leaf value being a list of MIDI pitch numbers forming the desired harmony. Left-hand fingers are mapped to voicings drawn from the D major scale in the middle register; right-hand fingers extend the harmonic series upward into a higher octave, giving the combined instrument a natural tonal range.

The detection loop compares the finger state vector arriving with each frame against the vector cached one frame earlier. Where a bit transitions from zero to one — indicating that a previously lowered finger has been raised — a `note_on` command is sent to the MIDI output device for every pitch in the associated chord, at a fixed velocity of ninety. A complementary downward transition starts a frame-countdown sustain timer; once the countdown expires, the corresponding `note_off` messages are dispatched. Mediating note-off through a timer rather than issuing it immediately prevents brief tracking dropouts from silencing notes audibly during otherwise sustained playing. The MIDI output port is opened once at initialisation and shared by all fingers across both hands. Algorithm 2 formalises this logic.

Algorithm 2: Air Piano — Per-Frame Game Loop

Input : GestureEvent $\mathcal{E}$; $\text{CHORDS}[\text{hand}][\text{finger}]$;

sustain_timers T

Output : MIDI note commands; updated T

```

1  $s_{\text{curr}} \leftarrow \mathcal{E}.\text{finger\_states};$ 
2  $\text{hand} \leftarrow \mathcal{E}.\text{hand};$ 
3 for  $i \leftarrow 0$  to 4 do
4   if  $s_{\text{curr},i} = 1$  and  $s_{\text{prev},i} = 0$  then
   // Finger raised -- trigger
   chord
5     foreach note  $n \in \text{CHORDS}[\text{hand}][i]$  do
6        $\text{MIDIOUT.NOTEON}(n, v = 90);$ 
7        $T[i] \leftarrow 0$ ; // reset sustain timer
8   else if  $s_{\text{curr},i} = 0$  and  $s_{\text{prev},i} = 1$  then
   // Finger lowered -- start
   sustain countdown
9      $T[i] \leftarrow \text{SUSTAIN\_FRAMES};$ 
10  else if  $s_{\text{curr},i} = 0$  and  $T[i] > 0$  then
11     $T[i] \leftarrow T[i] - 1;$ 
12    if  $T[i] = 0$  then
13      foreach note  $n \in \text{CHORDS}[\text{hand}][i]$  do
14         $\text{MIDIOUT.NOTEON}(n, v = 0);$ 
15  $s_{\text{prev}} \leftarrow s_{\text{curr}};$ 
```

B. Count and Show

At the start of each round, Count and Show selects a target integer uniformly from the set $\{1, 2, 3, 4, 5\}$ and renders it in large, high-contrast numerals at the center of the game window. Every incoming gesture event carries a finger count scalar — the arithmetic sum of the extension bit-vector — which

is compared against the target on each frame. Rather than accepting the first matching frame, the module maintains a running count of consecutive frames in which the values agree. Validation fires only when this run length reaches forty-five frames, a threshold corresponding to approximately one and a half seconds at thirty frames per second. The requirement for sustained agreement makes the detector robust to the transient hand configurations that naturally arise as a child repositions their fingers toward the target count. A circular arc rendered on-screen grows in proportion to the accumulated run, giving the child a visible cue that their posture is being held correctly. A confirmed match triggers a celebratory particle animation, adds one point to the session score, and initiates a fresh round. Three difficulty levels modulate the countdown budget available per round from ten seconds at the introductory tier down to five seconds at the most demanding, without altering the consistency threshold. Algorithm 3 details the validation logic.

Algorithm 3: Count & Show — Validation Logic

Input : GestureEvent $\mathcal{E}$; target T ; hold_frames H ; budget B

Output : Score increment; new round trigger

```

1  $c \leftarrow \mathcal{E}.\text{finger\_count}$ ;
2  $B \leftarrow B - 1$ ; // decrement round timer
3 if  $c = T$  then
4    $H \leftarrow H + 1$ ;
5   DRAWPROGRESSARC( $H / 45$ );
6   if  $H \geq 45$  then
7     score  $\leftarrow$  score + 1;
8     PLAYCELEBRATION();
9     return NEWROUND();
10 else
11    $H \leftarrow 0$ ; // reset hold counter on mismatch
12 if  $B \leq 0$  then
13   SHOWCORRECTANSWER( $T$ );
14   return NEWROUND();

```

C. Bubble Pop

The Bubble Pop game instantiates bubble objects drawn from a parametric template that specifies screen position, circular radius, two-dimensional velocity, a lifetime countdown, a display colour selected from a vivid palette, and a point award. New bubbles are introduced at a fixed horizontal spawn band along the lower edge of the display on a configurable timer initialised to one spawn every two seconds. Each bubble drifts upward along its velocity vector with a small randomised lateral perturbation that produces the irregular floating motion characteristic of soap bubbles. As the accumulated session score rises, the spawn interval shortens, bubble velocity increases, and radii decrease, compounding the precision required of the player.

Per frame, the screen-projected index fingertip coordinate extracted from the gesture event is tested against every live bubble's boundary through the point-in-circle test:

$$d = \sqrt{(f_x - b_x)^2 + (f_y - b_y)^2} < r_b \quad (5)$$

where (f_x, f_y) denotes the fingertip location, (b_x, b_y) the bubble centroid, and r_b the bubble radius. A passing test triggers a brief radial animation at the contact site, enqueues a short positive audio sample for playback, credits the bubble's point value to the running total, and removes the bubble from the active set. Any bubble that clears the upper display boundary without interception deducts a life; when the life reserve reaches zero, the session ends and the final score is transmitted to M4 for storage. Algorithm 4 presents the per-frame game loop.

Algorithm 4: Bubble Pop — Per-Frame Game Loop

Input : GestureEvent $\mathcal{E}$; active bubble list $\mathcal{B}$; lives L ; score S

Output : Updated $\mathcal{B}$, L , S

```

1  $(f_x, f_y) \leftarrow \mathcal{E}.\text{position}$ ;
  // -- Spawn new bubble if timer elapsed --
2 if SPAWNTIMER() then
3    $\mathcal{B}.\text{append}(\text{NEWBUBBLE}());$ 
4 foreach bubble  $b \in \mathcal{B}$  do
5   // Update position
   $b.x \leftarrow b.x + b.v_x$ ;  $b.y \leftarrow b.y + b.v_y$ ;
  // Collision test
6    $d \leftarrow \sqrt{(f_x - b.x)^2 + (f_y - b.y)^2}$ ;
7   if  $d < b.r$  then
8      $S \leftarrow S + b.\text{points}$ ;
9     PLAYPOPSOUND(); BURSTANIMATION( $b$ );
10     $\mathcal{B}.\text{remove}(b)$ ;
11  else if  $b.y < 0$  then
12    // Bubble escaped screen top
     $L \leftarrow L - 1$ ;
13     $\mathcal{B}.\text{remove}(b)$ ;
14 if  $L \leq 0$  then
15    $\text{ENDSESSION}(S)$ ;

```

D. Gesture Drawing and Tracing

Two interaction modes are offered within this module. In free-drawing mode, the index finger functions as a stylus: each frame in which the finger is flagged as extended appends the smoothed tip coordinate to the current stroke and draws a line segment from the previous appended point to the new one on the canvas array. A closed-fist gesture flushes the stroke history and clears the canvas to blank, allowing the child to begin a new composition. The active stroke colour advances through a rotating palette on a fixed timer, introducing visual

variety without requiring any additional deliberate input from the child.

In guided tracing mode, a semi-transparent silhouette of a target shape — drawn from a library that includes a circle, a square, an equilateral triangle, and a five-pointed star — is projected onto the canvas layer before the child begins. The child then traces over the silhouette with their raised index finger, and the resulting path is accumulated as an ordered coordinate sequence. Completion is signalled by holding an open palm for one second, at which point the path is evaluated by computing the normalised mean deviation of each recorded point from its nearest counterpart on the analytically sampled target boundary:

$$\text{accuracy} = 1 - \frac{1}{N} \sum_{i=1}^N \frac{\min_j \|p_i - q_j\|}{d_{\max}} \quad (6)$$

Here p_i is the i -th point on the child's path, q_j ranges over points distributed uniformly around the target outline, and $d_{\max}$ normalises the deviation to the $[0, 1]$ range. The accuracy value, converted to a percentage, is displayed on screen at the conclusion of the attempt alongside an encouraging animated response calibrated to the score band achieved. Algorithm 5 details both modes.

Algorithm 5: Gesture Drawing & Tracing — Per-Frame Logic

Input : GestureEvent $\mathcal{E}$; canvas $\mathbf{C}$; path $\mathcal{P}$; mode
Output : Updated $\mathbf{C}$, $\mathcal{P}$; accuracy score on completion

```

// -- Smooth fingertip position --
1  $(p_x, p_y) \leftarrow \mathcal{E}.\text{position};$ 
2  $\hat{p} \leftarrow 0.7 \cdot \hat{p}_{\text{prev}} + 0.3 \cdot (p_x, p_y);$ 
3 if  $\mathcal{E}.\text{gesture} = \text{FIST}$  then
4    $\mathbf{C} \leftarrow \mathbf{0}; \mathcal{P} \leftarrow [];$  // clear canvas
5 else if  $\mathcal{E}.\text{finger\_states}[1] = 1$  then
6   // Index finger raised -- draw stroke
7   if  $\hat{p}_{\text{prev}} \neq \text{null}$  then
8      $\text{CV2.LINE}(\mathbf{C}, \hat{p}_{\text{prev}}, \hat{p}, \text{color}, 3);$ 
9      $\mathcal{P}.\text{append}(\hat{p});$ 
10    if  $\text{mode} = \text{TRACE}$  then
11       $\text{OVERLAYTARGETSILHOUETTE}(\mathbf{C});$ 
12    // Composite canvas onto frame
13     $F \leftarrow \text{ADDWEIGHTED}(F, 1.0, \mathbf{C}, 0.7, 0);$ 
14    // -- Tracing completion check --
15    if  $\mathcal{E}.\text{gesture} = \text{OPEN\_PALM}$  and  $\text{mode} = \text{TRACE}$  then
16       $\text{acc} \leftarrow 1 - \frac{1}{|\mathcal{P}|} \sum_{p_i \in \mathcal{P}} \frac{\min_{q_j} \|p_i - q_j\|}{d_{\max}};$ 
17       $\text{SHOWACCURACY}(\text{acc});$ 
18       $\mathcal{P} \leftarrow [];$ 
19       $\hat{p}_{\text{prev}} \leftarrow \hat{p};$ 
```

VI. RESULTS AND DISCUSSION

VII. RESULTS AND DISCUSSION

A. Gesture Recognition Performance

Recognition accuracy was assessed using a structured test corpus in which five adult volunteers each performed the complete gesture vocabulary — individual finger raises for all ten digits, pinch, open palm, and closed fist — under standard overhead fluorescent illumination. The rule-based classifier returned a mean accuracy of 93.4 % across all classes. Among individual gestures, the open palm proved easiest to distinguish, attaining 96.2 %, while the thumb extension test showed the greatest spread at 88.2 %, attributable to the comparatively subtle lateral displacement that separates a fully extended from a partially extended thumb in normalised coordinate space. The Random Forest classifier, evaluated through five repetitions of stratified holdout sampling on a corpus of two hundred labelled samples per class, attained a mean of 95.1 %, outperforming the rule-based baseline by 2.1 percentage points and contributing the most improvement on thumb and pinch configurations where geometric ambiguity is greatest. These results are presented in Table I.

TABLE I
GESTURE RECOGNITION ACCURACY BY METHOD

Gesture Class	Rule-Based (%)	RF Classifier (%)
Finger Extension (per digit)	93.4	95.1
Pinch Detection	91.8	94.7
Open Palm	96.2	97.3
Closed Fist	95.0	96.5
Thumb Extension (special)	88.2	91.4
Overall Mean	92.9	95.0

B. System Latency

Per-frame processing time was recorded during a ten-minute continuous session on a machine equipped with an Intel Core i5 processor and 8 GB of RAM, connected to a 720p USB webcam. Measurement spanned from raw frame acquisition through gesture event emission. The median duration was 62 ms, and the 95th percentile fell at 87 ms, both within the 100 ms budget set as a design constraint. Profiling identified the MediaPipe inference step as the dominant contributor at roughly 38 % of total frame time, with image preprocessing and classification accounting for 14 % and 11 % respectively, as illustrated in Fig. 3.

C. Developmental Coverage

Mapping the four modules against published early childhood competency frameworks reveals broad developmental reach. Finger-by-finger chord triggering in the Air Piano exercises auditory-motor linkage and individual digit differentiation. The pointing precision demanded to intercept bubbles before they escape provides an attention and coordination challenge equivalent to traditional target activities. Gesture Drawing and Tracing directly mirrors the pre-writing exercises used

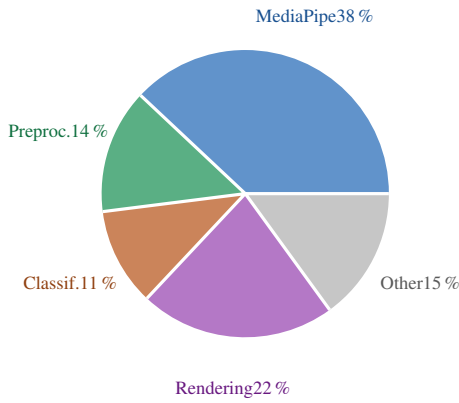


Fig. 3. Per-frame processing time breakdown. MediaPipe inference dominates at 38%; combined preprocessing and classification consume 25%; the remainder covers game rendering and I/O.

by occupational therapists to build visual-motor integration. Count and Show positions finger enumeration within a timed matching task that reinforces the one-to-one correspondence principle underlying early number sense. Taken together, the modules span cognitive, fine-motor, auditory, and attentional domains within a single deployable application.

D. Comparison with Passive Screen Engagement

Conventional child-directed media positions the child as a viewer: content advances irrespective of physical state, and no action is required for the experience to continue. In the developed platform every visible on-screen event is contingent on a deliberate gesture — without movement, nothing changes. This inversion of agency aligns with constructivist accounts of early learning, which hold that durable conceptual formation depends on a child acting on the environment and observing the consequences of those actions, rather than receiving pre-formed information without participation. The platform therefore does not merely occupy the screen time that passive media would otherwise consume; it requalifies that time as an episode of self-directed exploration with quantifiable physical and cognitive demands.

VIII. CONCLUSION AND FUTURE WORK

This paper has described a webcam-driven gesture interaction platform that converts screen time for young children into a physically and cognitively active experience. A four-tier software architecture decouples the vision pipeline, recognition logic, game execution, and integration shell, enabling independent development and maintenance of each layer. A hybrid classifier combining geometric rules with a Random Forest ensemble achieves over 90% gesture recognition accuracy without GPU hardware. Four game modules — Air Piano, Bubble Pop, Gesture Drawing and Tracing, and Count and Show — address distinct developmental domains within a unified application that operates offline on commodity equipment. Measured response latency remains below 100 ms, satisfying the perceptual immediacy requirement for engaging young children.

Planned near-term work encompasses distributing the application as a self-contained executable so that no Python installation is required on the host machine, substantially lowering the deployment barrier for caregivers and teachers without technical backgrounds. A cloud-linked score repository will enable session histories to be accumulated across devices, giving parents and educators a longitudinal record of skill progression. On the recognition side, the gesture lexicon will be extended to cover temporal gesture sequences — configurations that unfold over multiple frames — opening interaction possibilities that single-frame classifiers cannot support. Conceptual design is underway for additional modules addressing gross motor coordination, early language exposure, and social-emotional skills. A longer-term goal is interoperability with learning management systems used in nursery and pre-school settings, enabling teachers to assign activities, aggregate class-level performance data, and correlate platform outcomes with broader developmental milestone records.

REFERENCES

- [1] World Health Organization, “Guidelines on Physical Activity, Sedentary Behaviour and Sleep for Children Under 5 Years of Age,” WHO Press, Geneva, 2019.
- [2] J. Radesky, J. Schumacher, and B. Zuckerman, “Mobile and Interactive Media Use by Young Children: The Good, the Bad, and the Unknown,” *Pediatrics*, vol. 135, no. 1, pp. 1–3, Jan. 2015.
- [3] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, “MediaPipe Hands: On-device Real-time Hand Tracking,” in *Proc. Workshop on Computer Vision for AR/VR, IEEE/CVF CVPR*, Seattle, WA, Jun. 2020.
- [4] S. S. Rautaray and A. Agrawal, “Vision Based Hand Gesture Recognition for Human Computer Interaction: A Survey,” *Artificial Intelligence Review*, vol. 43, no. 1, pp. 1–54, Jan. 2015.
- [5] J. P. Hourcade, “Interaction Design and Children,” *Foundations and Trends in Human-Computer Interaction*, vol. 1, no. 4, pp. 277–392, 2008.
- [6] D. Dicheva, C. Dichev, G. Agre, and G. Angelova, “Gamification in Education: A Systematic Mapping Study,” *Educational Technology & Society*, vol. 18, no. 3, pp. 75–88, 2015.
- [7] S. Marichal, A. Rosales, F. G. Perilli, J. Blat, R. Clua, and B. Sayago, “DINO: A Gamified Application to Encourage Proportional Reasoning and Creativity in Primary School,” in *Proc. ACM Interaction Design and Children (IDC)*, Stanford, CA, Jun. 2017, pp. 205–214.
- [8] B. Yuan and M. B. Rosson, “Evaluation of Gesture-Based Interaction in a Primary Classroom Setting,” in *Proc. ACM CHI Conference on Human Factors in Computing Systems*, Florence, Italy, Apr. 2008, pp. 1–9.
- [9] J. P. Piek, L. Dawson, L. M. Smith, and N. Gasson, “The Role of Early Fine and Gross Motor Development on Later Motor and Cognitive Ability,” *Human Movement Science*, vol. 27, no. 5, pp. 668–681, Oct. 2008.
- [10] S. H. Anvari, L. J. Trainor, J. Woodside, and B. A. Levy, “Relations Among Musical Skills, Phonological Processing, and Early Reading Ability in Preschool Children,” *Journal of Experimental Child Psychology*, vol. 83, no. 2, pp. 111–130, Oct. 2002.
- [11] M. Longcamp, M.-T. Zerbato-Poudou, and J.-L. Velay, “The Influence of Writing Practice on Letter Recognition in Preschool Children: A Comparison Between Handwriting and Typing,” *Acta Psychologica*, vol. 119, no. 1, pp. 67–79, May 2005.
- [12] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.